

Three Anti-Patterns and Why Naming Patterns Matters — Hybrid Composition Coherence as a Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the three anti-patterns into which hybrid CKS deployments drift when adjacent components acquire unnamed positions, and the meta-commitment to naming patterns explicitly that makes architectural drift visible at the moment of practical decision.

Abstract

The hybrid systems composition specification commits CKS deployments to a three-positions architecture for adjacent AI components — Pattern A (input to a cell), Pattern B (derived view), Pattern C (separate concern). The integrating-frame note formalized the three positions; three companion notes specialized each pattern's operational components. This note closes the decomposition by formalizing what each pattern specification is *for*: identifying and preventing the three operationally common anti-patterns that arise when adjacent components acquire positions the architecture does not name. The three anti-patterns are *adjacent component as substitute for substrate* (Anti-pattern 1), *adjacent component as ungoverned writer* (Anti-pattern 2), and *hidden bidirectional coupling* (Anti-pattern 3). Each maps to violations of specific Pattern specifications and to violations of foundational architectural commitments the source paper defends. The standalone contribution of this note is the meta-commitment that anti-pattern visibility depends on pattern naming — that a team facing a practical engineering decision needs the three legitimate positions named explicitly to recognize a proposed shortcut as a pattern violation rather than as a sensible simplification, and that the architecture's silence on a fourth position is a constraint to be respected rather than a gap to be filled. This note closes both the hybrid-systems-composition decomposition (A2.91–A2.95) and, with it, the entire foundational A1 decomposition across all sixteen foundational commitments (A1.01 through A1.16).

1. Why the anti-patterns and naming meta-commitment need to be formalized as standalone

The integrating-frame note that opens this decomposition (the standalone treatment of A1.16's three-positions architecture) named the three legitimate positions an adjacent component may occupy relative to a CKS substrate. Three companion notes specialized each position: Pattern A as cell-consultation composition, Pattern B as derived-view composition, Pattern C as separate-concern composition. With those four notes in place, the three positions are operationally specified. What remains unspecified is the mirror question: what happens when a deployment has hybrid compositions but does not name positions explicitly? The answer — that the deployment drifts into one of three operationally common anti-patterns, and that the drift is visible only when the legitimate positions are named — is the standalone contribution of this note.

The motivating cases are deployments where adjacent components coexist with CKS substrates without explicit pattern naming. Three are common enough to be archetypal. (i) A vector index that began life as a Pattern B derived view becomes faster or more convenient than the substrate that produced it, then becomes the surface participants reach for first, then becomes the place new content is written, until the substrate is being reconstructed from the “derived” view; the source of truth has migrated. (ii) An autonomous agent backed by a fine-tuned model is granted write access to the substrate to streamline a workflow and writes coordination decisions directly without cell mediation; the writes appear in substrate state without orchestration-rule trace, without provenance for the consultation that produced them, and without the authority constraints a cell would have applied. (iii) A vector index reindexes substrate content automatically while also injecting embeddings back into substrate fields; neither direction passes through a cell or a rule, and the bidirectional coupling becomes critical infrastructure that no human-authored rule governs. Each scenario is operationally common, and each requires the anti-pattern to be identifiable before remediation is possible.

A second motivation is the strategic prior-art posture. The anti-patterns and the naming meta-commitment are consequential prior art because hybrid AI deployments commonly exhibit anti-pattern behaviors and because patentable derivations that focus on hybrid AI architectures with anti-pattern detection, architectural-drift prevention through pattern naming, or composition-coherence enforcement are substantially more defensibly contested when the anti-patterns and meta-commitment are publicly formalized as standalone.

A third motivation is the structural parallel to two earlier closing-specialization notes in the foundational decomposition. A2.85 closed A1.14's three-adjacencies decomposition by formalizing hybrid composition coherence on memory-location adjacencies; A2.90 closed A1.15's orchestration-layer-distinctions decomposition by formalizing composition pattern across orchestration layers. This note closes A1.16's hybrid-systems-composition decomposition by formalizing anti-patterns and composition coherence on the three-positions architecture. The three notes together specify a recurring meta-commitment to architectural-pattern naming that runs through all three composition-related foundational notes (A1.14, A1.15, A1.16). The parallel reinforces the meta-commitment as an architectural feature of CKS, not a property of any single composition framework.

A fourth motivation is the milestone framing. With this note complete, the foundational A1 decomposition is fully formalized across all sixteen foundational notes (A1.01 through A1.16). The closing section returns to this milestone explicitly.

2. The three anti-patterns, defined precisely

Each anti-pattern names a drift direction along which a deployment can move when adjacent components are present without explicit pattern naming. Each is identified by its mechanism, by the specific architectural commitments it violates, and by the remediation path that returns the deployment to a named position.

Anti-pattern 1 — Adjacent component as substitute for substrate. The deployment treats a RAG index, a vector database, or an external structured store as if it were the source of truth for coordination questions, gradually replacing or competing with the substrate. The drift typically begins as a Pattern B derived view per the Pattern B specification: the view is initially regenerated from substrate state, is non-authoritative on coordination questions, and absorbs no writes that flow back. Then the view becomes faster or more convenient than the substrate it derives from. Then participants reach for it first when answering coordination questions. Then the view becomes the place new coordination content is written. By the time the substrate is being reconstructed from the “derived” view, the source of truth has migrated.

The architectural commitments violated are: substrate-as-source-of-truth Category 1 per A2.43 (substrate authoritative for “what is the case”); substrate-as-source-of-truth Category 2 per A2.44 (substrate authoritative for “what is current”); Pattern B's third operational component per A2.93 (view non-authoritative on coordination questions); Pattern A's first operational component per A2.92 (substrate as primary source for cell consultations). The system-level failure mode is what the source paper §6.2 names *context rot*: substrate content participants thought authoritative has been compressed, summarized, or re-embedded out of fidelity with what the substrate carries.

The remediation path is to re-establish the substrate as primary source, treat the adjacent component as a Pattern B view with periodic regeneration from substrate state, and reconcile any orphaned writes that accumulated in the view back into substrate content under cell mediation. The remediation is rarely free; the longer the drift has run, the more substrate content has to be reconstructed.

Anti-pattern 2 — Adjacent component as ungoverned writer. The deployment allows an adjacent component — typically an autonomous agent backed by a fine-tuned model — to write coordination state outside cell mediation and orchestration-rule authorization. The component's writes appear in the substrate without orchestration-rule trace per A2.04, without provenance per A2.40 for the consultation that produced them, and without the authority constraints per A2.47 a cell would have applied. The mechanism is structural: the agent is given write credentials, a routing path, or a tool that interacts with substrate state directly, and the deployment treats the agent's writes as architecturally equivalent to cell writes.

The architectural commitments violated are: AI-as-substrate-mediator per A1.04 §4.2 (the LLM in the autonomous agent does not operate as a substrate mediator); human-governed per A1.01 §3.1 and §3.3 (humans do not govern the writes through orchestration rules); Pattern A's third operational component

per A2.92 (provenance crosses the component boundary cleanly); Pattern A's fourth operational component per A2.92 (cell outputs remain orchestration-rule-governed); Property B of the mediator decomposition per A2.20 (LLM writes occur under orchestration rules); Property E of the mediator decomposition per A2.23 (LLM outputs that affect substrate state are recorded with attribution); Composition Requirement D per A2.79 (AI-as-substrate-mediator preserved at every composition layer). If the component was nominally Pattern C, the anti-pattern additionally violates Pattern C's third operational component per A2.94 (component does not exercise governance authority over coordination state).

The architectural test is simple and load-bearing: every substrate write must be traceable to a cell and its rule; writes that are not have come from outside the architecture. The remediation path is to route the agent's contribution through a cell with an authorizing rule. The agent becomes the cell's input or consultant under Pattern A; the cell becomes the writer; provenance for the agent's consultation crosses the component boundary cleanly; and the cell's write carries the rule trace the architecture requires.

Anti-pattern 3 — Hidden bidirectional coupling. The substrate and an adjacent component update each other through automated processes that no human-authored orchestration rule governs. A typical instance: a vector index that reindexes substrate content automatically when substrate state changes while also injecting embeddings back into substrate fields when index state changes. Neither direction passes through a cell or a rule. The mechanism is operational rather than architectural — the coupling is typically introduced as a deployment-level convenience by infrastructure that pre-dates the substrate's CKS framing or by vendor-bundled tooling that operates “below” the cell layer.

The architectural commitments violated are: substrate-cell boundary per A1.02 §2.1 (the boundary is silently bypassed in both directions); path retraceability per A1.07 (the trace that would have recorded “this write happened because rule R authorized cell C” no longer exists for the inbound direction); Pattern B's fourth operational component per A2.93 (writes do not flow back from the view to the substrate); Pattern A's third operational component per A2.92 (provenance crosses the component boundary cleanly).

The remediation path is to bring each direction of update under an orchestration rule. The outbound direction (substrate change triggers index rebuild) becomes Pattern B, with the rebuild operating as a derived-view regeneration that does not change substrate state. The inbound direction (index informs substrate content) becomes Pattern A, with a cell consulting the index under an authorizing rule and writing substrate state as the cell's output. The coupling is unwound into two named compositions, each operationally enforceable.

3. The naming meta-commitment, defined precisely

The three anti-patterns share a common failure mode: an adjacent component has acquired a position the architecture does not name, and the commitments degrade where the position is unnamed. The naming meta-commitment is the architectural specification that prevents the failure. It has four operational components.

Three operationally common anti-patterns are recognizable. The anti-patterns are not theoretical — they are commonly observed in real hybrid AI deployments in 2024–2026. Substrate substitution,

ungoverned writing, and hidden bidirectional coupling are the three drift directions hybrid compositions take when patterns are not named. Recognizability requires that the three drifts have names, that the names map to specific architectural violations, and that a deployment review can identify which drift, if any, a specific composition exhibits.

Each anti-pattern corresponds to specific architectural commitment violations. Anti-pattern 1 violates A2.43, A2.44, and Pattern B's non-authoritativeness commitment per A2.93. Anti-pattern 2 violates A1.04, A1.01, Pattern A's provenance and rule-governance commitments per A2.92, the mediator-decomposition properties per A2.20 and A2.23, and Composition Requirement D per A2.79. Anti-pattern 3 violates A1.02, A1.07, Pattern B's no-write-flow-back commitment per A2.93, and Pattern A's clean-provenance commitment per A2.92. The mappings are precise rather than illustrative — each anti-pattern is exactly the violation of the named commitments, not a fuzzy resemblance to it.

Naming the three legitimate positions makes anti-pattern drift visible at the moment of practical decision. A team facing the question “should we let the agent write to the substrate directly to skip the cell?” needs a frame that surfaces the commitment the shortcut would violate. *Pattern A says the agent's input goes through the cell; the cell's rule authorizes the write* is that frame, and the proposed shortcut is identifiable as a Pattern A bypass. Without the frame, the same decision reads as a sensible engineering simplification and is taken without recognizing what it costs. Visibility is the operational consequence of naming; without names, the architectural commitments are not visible at the moment a practical decision is made.

The architecture's silence on a fourth position is a constraint, not an invitation. The three-positions architecture is exhaustive within the source paper's architectural commitments. A team that discovers a desired composition fits none of the three positions cleanly is being informed that the desired composition is one the source paper does not support — not that the source paper has a gap to be filled by analogy. Respecting the constraint is what makes hybrid composition defensible without introducing new commitments beyond the source paper.

The four components together define the naming meta-commitment architecturally. A deployment that satisfies all four has the meta-commitment in the architectural sense.

4. What the anti-patterns and naming meta-commitment do NOT claim

The standalone treatment is precise about what the meta-commitment is. It is equally precise about what it is not.

The meta-commitment does not claim that hybrid compositions are inherently risky. The architectural commitment is to coherence when hybrids exist, not to hybrid avoidance. Hybrid deployments are the common case; the anti-patterns are what hybrids drift toward when patterns are not named explicitly, not what hybrids are.

It does not foreclose architectural evolution. A specific component may transition between patterns over a deployment lifecycle — a Pattern C component may take on coordination-relevant work and need to be re-evaluated as Pattern A; a Pattern B view may need to be retired or rebuilt as the substrate's structure evolves. Composition Requirement E per A2.80 (human-selective composition) names the

right under which such transitions occur; the meta-commitment requires that transitions are recognized as transitions, with the new pattern's requirements operationally enforced.

It does not specify implementation patterns for anti-pattern detection. Implementations may use architectural reviews, deployment audits, operational monitoring, automated checks, or any combination; the architectural commitment is to anti-pattern recognizability through pattern naming, not to specific detection implementations.

It does not require all anti-patterns to be eliminated immediately on detection. Some anti-patterns may be operationally entrenched and require deliberate remediation; the architectural commitment is to recognizability and remediability under Composition Requirement E, not to instant elimination.

It does not foreclose new anti-patterns. The three named here are the most common anti-patterns observed in 2024–2026 hybrid AI deployments; future architectural work may identify additional anti-patterns. The meta-commitment to naming patterns explicitly is precisely what makes new anti-patterns identifiable as they emerge.

It does not require all hybrid compositions to use all three patterns. A deployment may use only Pattern A, only Pattern B, only Pattern C, or any subset; the architectural commitment is that for each adjacent component present, the position it occupies is identifiable and the requirements of that position are met.

5. What the anti-patterns and meta-commitment are NOT

The meta-commitment is commonly conflated with four adjacent governance approaches. Each of these is a real and reasonable commitment in some other framework; conflating any with the meta-commitment produces a misreading of CKS hybrid composition.

Not generic AI governance. Generic AI governance frameworks address operational concerns (model selection, deployment authorization, ongoing monitoring) and tend to operate at the deployment-policy level. The anti-patterns and meta-commitment are different: they specify architectural-pattern violations and the meta-commitment to pattern naming. Generic AI governance may operationally complement the meta-commitment but does not substitute for it; a deployment with strong AI governance practices and no pattern naming will still drift into anti-patterns.

Not vendor-bundled best practices. Vendor-bundled best practices (recommended deployment patterns, vendor-blessed architectures) operate at the vendor-recommendation level. The meta-commitment is different: it specifies architectural-pattern requirements regardless of vendor recommendations. Vendor practices may operationally conflict with or support the meta-commitment depending on architectural alignment, and several common vendor-bundled compositions are recognizable as Anti-pattern 3 (hidden bidirectional coupling between substrate-style stores and vector indexes shipped as integrated tooling).

Not deployment quality metrics. Deployment quality metrics (uptime, latency, accuracy, retrieval quality) operate at the operational-measurement level. The meta-commitment is different: it specifies architectural-pattern coherence regardless of operational performance. A deployment may have excellent operational metrics while violating the meta-commitment if patterns are not named explicitly; conversely, a deployment with the meta-commitment in place may have any operational profile.

Not architectural style guides. Architectural style guides specify recommended approaches at the design-recommendation level. The meta-commitment is different: it specifies load-bearing architectural commitments that compositions must satisfy to be CKS-coherent. Style guides may operationally support the meta-commitment when their recommendations align with the architectural commitments, but a style guide is advisory where the meta-commitment is constraining.

6. Why the anti-patterns and meta-commitment are load-bearing for downstream commitments

The meta-commitment is load-bearing for several CKS commitments.

For the integrating hybrid-systems-composition specification per A1.16 and the integrating-frame note: the meta-commitment is the operational consequence of the three-positions architecture. Without it, the three positions would be specifications without enforcement, identifiable in principle but not operationally consequential.

For the three pattern specifications per A2.92, A2.93, and A2.94: the anti-patterns operationalize each pattern's failure modes. Pattern A's specification gains its operational force from Anti-pattern 2's identifiability as a Pattern A violation; Pattern B's specification gains its operational force from Anti-pattern 1's identifiability as a Pattern B drift and Anti-pattern 3's identifiability as a Pattern B violation; Pattern C's specification gains its force from the negative scope it preserves against Anti-pattern 2.

For the structural parallel to A2.85 and A2.90: all three notes formalize a meta-commitment to architectural-pattern naming on a different composition surface (memory-location adjacencies, orchestration-layer distinctions, three-positions architecture). Together they specify the recurring meta-pattern in CKS architecture under which boundary-naming enables coherent composition.

For the substrate-as-source-of-truth commitments per A2.43–A2.47: Anti-pattern 1 directly violates the source-of-truth commitments. The meta-commitment ensures the violation is recognizable rather than gradual and unnoticed.

For the AI-as-substrate-mediator commitment per A1.04 and the mediator decomposition A2.18–A2.23: Anti-pattern 2 directly violates the mediator commitment. The meta-commitment ensures the violation is recognizable as a Pattern A bypass rather than as a sensible deployment shortcut.

For the path-retraceability commitment per A1.07 and the retraceability decomposition A2.35–A2.41: Anti-pattern 3 breaks the retraceable trail. The meta-commitment ensures the bypass is recognizable as a violation rather than as infrastructure plumbing operating “below” the architecture.

For the substrate-cell-boundary commitment per A1.02 and the substrate-cell decomposition A2.08–A2.12: Anti-pattern 3 silently bypasses the boundary. The meta-commitment ensures the bypass is detectable as such.

For the composition requirements per A1.13 and the composition-requirements decomposition A2.75–A2.80: the five composition requirements operate at pattern boundaries. The anti-patterns

correspond to violations of specific requirements (Anti-pattern 2 violates Requirement D; transitions between patterns invoke Requirement E); the meta-commitment ensures requirement violations are recognizable through pattern naming.

7. Failure modes that violate the meta-commitment

Each of the following names a way an implementation can fail by allowing compositions to acquire unnamed positions.

Compositions exist without explicit pattern naming. The implementation has hybrid compositions but does not explicitly identify which adjacent components occupy which patterns. The compositions exist operationally but are unnamed architecturally; anti-patterns are not detectable because patterns are not named.

Pattern names are nominal but not enforced. The implementation labels components by pattern in documentation but does not enforce the pattern's operational components. The naming is performative; the architectural commitments operate without the patterns' requirements actually applying.

New compositions adopted without pattern evaluation. The implementation adds adjacent components to deployments without evaluating which pattern they should occupy. The components acquire ad-hoc positions that may or may not fit the three-positions architecture, and anti-patterns emerge as components drift into unnamed positions.

Pattern transitions are not recognized as transitions. The implementation has adjacent components that change scope or role over deployment lifecycle, but the transitions are not recognized as architectural events. A Pattern C component scope-creeping into coordination work is not re-evaluated as Pattern A; a Pattern B view becoming authoritative is not recognized as drift toward Anti-pattern 1.

Anti-patterns operationally entrenched. The implementation has anti-patterns that have become operationally essential — substitution where the derived view is now unmovable, ungoverned writes where the agent is now relied on, bidirectional coupling where the automated process is now critical. The meta-commitment to remediation under Composition Requirement E per A2.80 is bypassed in favor of operational continuity.

Anti-pattern detection treated as feature, not commitment. The implementation treats anti-pattern detection as an optional feature rather than as the operational consequence of the meta-commitment to pattern naming. Detection is performed inconsistently; anti-patterns persist without architectural recognition.

Fourth-position-by-analogy. The implementation invents a fourth position by analogy when desired compositions do not fit the three-positions architecture. The architecture's silence is treated as a gap to be filled rather than as a constraint to be respected; new anti-patterns emerge as ad-hoc fourth positions are normalized.

Vendor-driven pattern violations accepted. The implementation accepts vendor-bundled compositions that violate one or more patterns because the vendor practice is operationally established. The meta-commitment to pattern coherence is sacrificed for vendor alignment.

Operational complexity treated as justification for pattern violations. The implementation argues that specific compositions are too operationally complex to fit the three-positions architecture cleanly and accepts pattern violations as necessary trade-offs. The architectural commitment is sacrificed for operational simplicity.

Pattern-naming reduced to documentation. The implementation maintains pattern names in architectural documentation, but the names do not correspond to operational architecture. The meta-commitment is performative; the architectural reality drifts independently of the documentation.

8. Operational test

A deployment satisfies the anti-patterns-and-naming-meta-commitment if and only if all of the following are true at all times during the deployment's existence.

1. Each adjacent component in the deployment is positioned in Pattern A, Pattern B, or Pattern C explicitly per A2.92, A2.93, or A2.94 — no component occupies an unnamed position.
2. The position is operationally enforced — the pattern's operational components are satisfied per the corresponding pattern specification, not merely claimed in documentation.
3. Anti-patterns are recognizable through pattern naming — a deployment review can identify whether any component exhibits Anti-pattern 1 (substrate substitute), Anti-pattern 2 (ungoverned writer), or Anti-pattern 3 (hidden bidirectional coupling) by reference to the pattern specifications.
4. Anti-patterns when detected are remediated through pattern realignment — components that have acquired anti-pattern behavior are re-evaluated under the appropriate pattern per Composition Requirement E per A2.80; anti-patterns are not allowed to persist as accepted operational reality.
5. The architecture's silence on a fourth position is respected — compositions that do not fit the three-positions architecture cleanly are not implemented as fourth-position-by-analogy; they are recognized as compositions the source paper does not support.
6. Pattern transitions are recognized as architectural events — components that change scope or role over deployment lifecycle have their pattern position re-evaluated and operationalized accordingly.

A deployment that fails any of (1)–(6) does not satisfy the meta-commitment in the architectural sense, even if it operationally appears to use the three patterns.

9. The one-sentence test

If a deployment can identify, for each adjacent component, which of the three legitimate positions it occupies, and the requirements of that position are operationally enforced, the meta-commitment is satisfied; if any component occupies an unnamed position or the named position's requirements are not enforced, an anti-pattern is either present or imminent and the architectural commitments are degrading.

The one-sentence test names the most operationally distinctive property — per-component pattern naming with operational enforcement — for any specific deployment. The four-component

specification in §3 plus the operational test in §8 provide the full architectural definition for cases requiring detailed analysis; the one-sentence test is the form analysts and reviewers can apply quickly.

10. Why naming the meta-commitment as standalone matters, and foundational A1 decomposition closing observations

Implementations under pressure to deliver hybrid AI architectures consistently drift into the three anti-patterns named in §2. The drift is steady because hybrid AI is operationally attractive — multiple components offer combined capabilities that no single component delivers — and because commercial AI products typically combine substrate-style and adjacent-component layers without explicit pattern naming, presenting the combination as a unified offering rather than as a hybrid composition. Audiences understand the description “the system uses RAG and a fine-tuned LLM” more readily than “Pattern A consultation with provenance crossover and rule-governed cell output.” The pull toward the simpler description is steady.

Implementations that drift away from the meta-commitment produce systems where compositions silently acquire anti-pattern behaviors. The downstream consequences manifest as substrate-authority erosion (Anti-pattern 1 produces context rot), AI-mediator failures (Anti-pattern 2 produces ungoverned writes), retraceability failures (Anti-pattern 3 breaks the retraceable trail), and architectural-commitment failures across the foundational A1 commitments. Each can be reframed as a violation of a named commitment once the patterns are named; without naming, the failures present as quality issues, performance issues, or operational complexity rather than as architectural drift.

Naming the anti-patterns and meta-commitment as a standalone architectural commitment — with the three anti-patterns in §2, the four operational components in §3, the limitations in §4, the four adjacent-approach distinctions in §5, the load-bearing connections in §6, the ten failure modes in §7, the operational test in §8, and the one-sentence test in §9 — gives downstream readers a precise specification of why the pattern naming the prior specializations provide matters operationally. The standalone form is what makes the meta-commitment defensible as prior art independent of any single pattern specification.

With this note complete, the hybrid-systems-composition decomposition is fully formalized. The integrating-frame note established the three-positions architecture; Pattern A was specialized as cell-consultation composition; Pattern B as derived-view composition; Pattern C as separate-concern composition; and this anti-patterns-and-meta-commitment specification specifies why pattern naming matters operationally. Together the five notes constitute the operational decomposition of A1.16.

With this note complete, additionally, the foundational A1 decomposition is fully formalized across all sixteen foundational notes (A1.01 through A1.16). The decomposition has operationalized each foundational architectural commitment from the source paper as a series of standalone derivation notes, each of which can be defended, implemented, and tested independently of the others. Per the master plan, the series next moves into subsequent Series A phases — the remainder of the operational variants, the dedicated anti-pattern formalizations under Phase A3, the composition-pair treatments under Phase A4, the operational-tests-as-standalone treatments under Phase A5, and the boundary-case treatments under Phase A6 — each of which builds on the foundational decomposition this note

completes.

The structural parallel between A2.85 (closing-specialization within A1.14's three-adjacencies decomposition), A2.90 (closing-specialization within A1.15's orchestration-layer-distinctions decomposition), and this note (closing-specialization within A1.16's hybrid-systems-composition decomposition) is the recurring meta-commitment to architectural-pattern naming that runs through all three composition-related foundational notes. The meta-commitment is what the foundational A1 decomposition has now operationalized at full depth: the three legitimate positions a composition may occupy on each surface have been named, and the architectural drift the silence on a fourth position would otherwise admit has been bounded by the act of naming. The downstream Series A phases that follow this note will operate in the operational space the foundational decomposition has now fully framed.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Three Anti-Patterns and Why Naming Patterns Matters — Hybrid Composition Coherence as a Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.